Практическая работа: Проектируем логику службы каршеринга электрокаров

Предметная область: платформа каршеринга электромобилей

Вы — команда разработки платформы аренды электрокаров. Схема БД уже спроектирована и заполнена тестовыми данными. Ваша задача — реализовать **бизнес-логику**, но инструментарий (триггер, процедура или функция) вы выбираете **сами**.

Что сдать в отчёте

1. **SQL-скрипты** всех 8 реализаций — каждый объект в отдельном блоке с заголовком.
2. **Таблица обоснований** — заполненная, с аргументацией по каждому требованию.
3. **Тестовые вызовы** — для каждого требования минимум один вызов, демонстрирующий корректную работу и один — демонстрирующий обработку ошибки.
4. **Скриншот** требования 8: попытка назначить авто в статусе Maintenance через прямой UPDATE dbo.Rentals SET CarId = 5 WHERE RentalId = 4 — с сообщением об ошибке.

Схема базы данных

```
CREATE TABLE dbo.Cars (
   CarId           INT IDENTITY(1,1) PRIMARY KEY,
   VinNumber       NVARCHAR(50)  NOT NULL UNIQUE,
   Model           NVARCHAR(50)  NOT NULL,
   Year            INT NOT NULL,
   MaxRangeKm      INT NOT NULL,              -- запас хода на полном заряде
   BatteryLevel    INT NOT NULL DEFAULT 100,    -- 0–100%
   CurrentMileage  DECIMAL(10,2) NOT NULL,     -- пробег в км
   Status          NVARCHAR(20) NOT NULL DEFAULT 'Available'
      CHECK (Status IN ('Available','Rented','Maintenance','Charging','Accident')),
   LastServiceDate  DATE NULL,
   NextServiceKm    DECIMAL(8,2) NOT NULL
);

CREATE TABLE dbo.Drivers (
   DriverId        INT IDENTITY(1,1) PRIMARY KEY,
   FullName        NVARCHAR(100) NOT NULL,
   LicenseNumber   NVARCHAR(20)  NOT NULL UNIQUE,
   LicenseExpiry   DATE NOT NULL,
   TotalRents      INT NOT NULL DEFAULT 0,
   TotalDistance   DECIMAL(10,2) NOT NULL DEFAULT 0,
   AccidentCount   INT NOT NULL DEFAULT 0,
   IsActive        BIT NOT NULL DEFAULT 1
);

CREATE TABLE dbo.Customers (
   CustomerId      INT IDENTITY(1,1) PRIMARY KEY,
   FullName        NVARCHAR(100) NOT NULL,
   Phone           NVARCHAR(20)  NOT NULL,
   Email           NVARCHAR(100) NULL,
   City            NVARCHAR(50)  NOT NULL
);

CREATE TABLE dbo.Rentals (
   RentalId        INT IDENTITY(1,1) PRIMARY KEY,
   CustomerId      INT NOT NULL,
```

```
    CarId           INT NOT NULL,
    DriverId        INT NOT NULL,
    StartAt         DATETIME NOT NULL,
    EndAt           DATETIME NULL,
    StartOdometer   DECIMAL(10,2) NOT NULL,
    EndOdometer     DECIMAL(10,2) NULL,
    DistanceCovered DECIMAL(10,2) NULL,         -- км
    BaseCost        DECIMAL(10,2) NOT NULL,     -- базовая стоимость
    ExtraCost       DECIMAL(10,2) NOT NULL DEFAULT 0, -- доп. штрафы/надбавки
    TotalCost       DECIMAL(10,2) NULL,
    Status          NVARCHAR(20) NOT NULL DEFAULT 'Active'
        CHECK (Status IN ('Active','Completed','Cancelled','Disputed')),
    CreatedAt       DATETIME NOT NULL DEFAULT GETDATE(),
    CONSTRAINT FK_Rentals_Customer FOREIGN KEY (CustomerId) REFERENCES
dbo.Customers(CustomerId),
    CONSTRAINT FK_Rentals_Car FOREIGN KEY (CarId) REFERENCES dbo.Cars(CarId),
    CONSTRAINT FK_Rentals_Driver FOREIGN KEY (DriverId) REFERENCES dbo.Drivers(DriverId)
);

CREATE TABLE dbo.ChargingLogs (
    LogId           INT IDENTITY(1,1) PRIMARY KEY,
    CarId           INT NOT NULL,
    ChargingStart   DATETIME NOT NULL,
    ChargingEnd     DATETIME NULL,
    EnergyConsumedKwh DECIMAL(6,2) NULL,          -- сколько кВт·ч «залито»
    ChargeFrom      INT NULL,                     -- уровень до зарядки
    ChargeTo        INT NULL,                     -- уровень после зарядки
    Location        NVARCHAR(100) NULL,
    CONSTRAINT FK_CL_Car FOREIGN KEY (CarId) REFERENCES dbo.Cars(CarId)
);

CREATE TABLE dbo.RentalStatusLog (
    LogId           INT IDENTITY(1,1) PRIMARY KEY,
    RentalId        INT NOT NULL,
    OldStatus       NVARCHAR(20) NULL,
    NewStatus       NVARCHAR(20) NOT NULL,
    ChangedAt       DATETIME NOT NULL DEFAULT GETDATE(),
    ChangedBy       NVARCHAR(100) NOT NULL DEFAULT SYSTEM_USER,
    CONSTRAINT FK_RSL_Rental FOREIGN KEY (RentalId) REFERENCES dbo.Rentals(RentalId)
);
```

Тестовые данные

```
INSERT dbo.Cars (VinNumber, Model, Year, MaxRangeKm, BatteryLevel, CurrentMileage, Status,
LastServiceDate, NextServiceKm) VALUES
(N'WBA12345678901234', N'BMW i3', 2021, 260, 82, 15200.5, N'Available', '2025-08-01', 16000),
(N'WBA54321987654321', N'BMW i3', 2022, 260, 45, 9800.2,  N'Available', '2025-07-10', 10500),
(N'VIN-TESLA-001',    N'Tesla Model 3', 2023, 528, 95, 3200.0,  N'Available', '2025-09-01', 5000),
(N'VIN-TESLA-002',    N'Tesla Model 3', 2020, 425, 12, 41200.7, N'Charging',  '2025-08-15', 42000),
(N'VIN-NISSAN-LEAF',  N'Nissan Leaf', 2019, 240, 70, 58000.3, N'Maintenance', '2025-06-01', 60000);

INSERT dbo.Drivers (FullName, LicenseNumber, LicenseExpiry, TotalRents, TotalDistance,
AccidentCount, IsActive) VALUES
```

```
(N'Игорь Петров',    N'DL-2024-101', '2027-12-31', 35, 1200.5, 0, 1),
(N'Ольга Смирнова',   N'DL-2024-102', '2026-11-20', 52, 2100.0, 1, 1),
(N'Дмитрий Волков',   N'DL-2023-103', '2025-10-10', 8,  320.2,  2, 1),
(N'Анна Кузнецова',   N'DL-2025-104', '2028-03-05', 12, 450.0,  0, 1);

INSERT dbo.Customers (FullName, Phone, Email, City) VALUES
(N'Сергей Иванов',    N'+7-900-111-22-33', N'sergey@example.com', N'Москва'),
(N'Елена Морозова',   N'+7-900-222-33-44', N'elena@example.com',  N'Москва'),
(N'Максим Соколов',   N'+7-900-333-44-55', NULL,                  N'Санкт-Петербург'),
(N'Ирина Лебедева',   N'+7-900-444-55-66', N'irina@example.com', N'Казань');

INSERT dbo.Rentals (CustomerId, CarId, DriverId, StartAt, EndAt, StartOdometer, EndOdometer,
DistanceCovered, BaseCost, ExtraCost, TotalCost, Status, CreatedAt) VALUES
(1, 1, 1, '2025-09-17 09:00', '2025-09-17 14:30', 15200.5, 15245.2, 44.7, 2800.00, 0, NULL, 'Completed',
'2025-09-17 08:55'),
(2, 3, 2, '2025-09-17 11:00', '2025-09-17 13:15', 3200.0,  3285.5, 85.5, 4200.00, 0, NULL, 'Completed',
'2025-09-17 10:55'),
(3, 2, 3, '2025-09-18 08:00', NULL,              9800.2,  NULL,    NULL, 1800.00, 0, NULL, 'Active',
'2025-09-18 07:55'),
(4, 1, 4, '2025-09-18 10:30', NULL,              15245.2, NULL,    NULL, 2500.00, 0, NULL, 'Active',
'2025-09-18 10:25');

INSERT dbo.ChargingLogs (CarId, ChargingStart, ChargingEnd, EnergyConsumedKwh, ChargeFrom,
ChargeTo, Location) VALUES
(2, '2025-09-17 20:00', '2025-09-18 00:30', 32.5, 45, 98, N'Зарядная станция ВДНХ'),
(4, '2025-09-18 09:00', NULL, NULL, 12, NULL, N'Подземная парковка ТЦ');
GO
```

Бизнес-требования

Вам даны **8 бизнес-требований**, описанных на языке заказчика. Для каждого требования вы должны:

1. **Самостоятельно выбрать** тип программируемого объекта (триггер, процедура или функция).
2. **Реализовать** логику.
3. **Обосновать** выбор в таблице ниже.

!!! В требованиях **не указано**, какой инструмент использовать. Это часть задачи — решить самим. Некоторые требования можно решить несколькими способами. Оценивается не только реализация, но и аргументация.

Требование 1. «Автоматический журнал статусов аренды»

Каждый раз, когда статус аренды меняется (например, Active → Completed → Cancelled), система должна автоматически записывать в RentalStatusLog кто, когда и с какого статуса на какой перевёл аренду. Ручная запись в журнал недопустима — оператор может забыть.

Требование 2. «Расчёт примерного запаса хода»

Менеджеру нужен показатель: примерный запас хода (в км) для текущего уровня батареи. Он считается как MaxRangeKm * (BatteryLevel / 100). Этот показатель должен быть доступен в любом SELECT по автомобилям, в отчётах и дашбордах.

Требование 3. «Оформление аренды авто»

Оператор оформляет аренду: выбирает клиента, автомобиль, водителя, фиксирует начальный пробег. Система должна: проверить, что автомобиль в статусе Available; проверить, что у водителя не истекла лицензия; проверить, что заряда хватит минимум на 30% от ожидаемого пробега (ожидание = MaxRangeKm * 0.3); назначить аренду, сменить статус авто на Rented, зафиксировать StartOdometer и StartAt. Если что-то не подходит — ошибка с понятным текстом. Процесс должен быть атомарным.

Требование 4. «Фиксация завершения аренды и расчёт стоимости»

Когда аренда завершена, оператор отмечает это в системе. Процесс должен: проверить, что аренда в статусе Active (нельзя закрыть то, что уже закрыто); установить EndAt, EndOdometer; посчитать DistanceCovered; рассчитать TotalCost = BaseCost + ExtraCost (а в будущем — с учётом пробега и перерасхода батареи); обновить CurrentMileage у автомобиля; увеличить TotalRents, TotalDistance у водителя; если были аварии — увеличить AccidentCount. Процесс атомарный, с обработкой ошибок.

Требование 5. «Авто-перевод в зарядку при низком заряде»

Если уровень заряда автомобиля падает ниже 20% (в результате завершения аренды), система должна **автоматически** перевести автомобиль в статус Charging. Оператор не должен следить за этим вручную.

Требование 6. «Рейтинг надёжности водителя»

Для аналитики и отчётов нужен **рейтинг надёжности водителя** — число от 0 до 100. Формула: ((TotalRents - AccidentCount) / TotalRents) * 100 (если TotalRents > 0, иначе 0). Значение должно быть доступно в любом SELECT по водителям.

Требование 7. «Фиксация зарядки и восстановление заряда»

Когда зарядка автомобиля завершена (оператор фиксирует ChargingEnd и ChargeTo), система должна: автоматически списать «расход» из виртуального баланса энергии (не требуется), но главное — **восстановить** BatteryLevel автомобиля до ChargeTo; зафиксировать EnergyConsumedKwh как разницу ChargeTo - ChargeFrom в процентах (для упрощения считаем 1% = 1 кВт·ч). Если ChargeTo меньше ChargeFrom — ошибка. Процесс атомарный.

Требование 8. «Защита от аренды неисправного авто»

Система должна **гарантировать** на уровне БД, что автомобиль в статусах Maintenance, Accident или Charging **никогда** не будет назначен на новую аренду — ни через процедуру, ни через прямой UPDATE, ни через приложение. Если кто-то попытается — операция должна быть отклонена с понятным сообщением.

Таблица обоснований (обязательная часть отчёта)

Заполните эту таблицу для каждого требования. Оценивается аргументация, а не только реализация.

№	Выбранный объект	Почему этот, а не другой?	Почему альтернативы не подходят?
1			
2			
3			
4			
5			
6			
7			
8			

Подсказка для выбора

Вопрос себе	**Если ответ «да» →**
Логика должна срабатывать **автоматически** при любом изменении данных, даже прямым UPDATE?	**Триггер**
Это **многократное действие** с валидацией, транзакцией и обработкой ошибок?	**Процедура**
Это **расчёт**, который нужно использовать внутри SELECT, WHERE, отчётов?	**Функция**
Несколько ответов «да» одновременно?	Думайте глубже. Часть логики может быть в триггере, часть — в процедуре.